Software Requirements Specification

Brain Training - Bookstore Purchase & Home Delivery Management System

Estándar: IEEE Std. 830-1998 (Version Adapted for a Prototype)
Fecha: October 2026
Prepared by team CtrlZ: Andres Daquilema, Lino Diaz, Mateo Dominguez

1. Introduction

1.1. Purpose

The purpose of this document is to define the specifications, features, non-functional requirements, and constraints of the "Brain Training" Bookstore System. This document serves as the primary technical reference for:

1. **Software Development Team:** For designing domain models, structuring core business logic, implementing Clean Code / Refactored Functions, and preparing serialization/reporting modules.
2. **Bookstore Staff & Admins:** To manage book catalogs, inventory levels, and order distribution.
3. **Delivery Drivers:** To view assigned orders, inspect delivery addresses, and update package delivery statuses.

1.2. System Scope & Project Phases

The software is officially named **"Brain Training BookStore"**. To ensure progressive delivery, the system development is organized in two main phases:

- **Phase 1 (Current Implementation - Console Prototype & Data Persistence):**
  - ➢ **Refactored CLI Interface:** Structured role selection menu (Admin, Customer, Delivery Driver) implemented through Clean Code refactoring functions.
  - ➢ **Customer Module:** Catalog browsing, book searching, placing orders, calculating totals, specifying delivery addresses, choosing payment methods, and tracking order delivery status.
  - ➢ **Admin Module:** Full management of authors, publishers, categories, books, stock reordering alerts, and order overview.
  - ➢ **Delivery Driver Module:** Viewing assigned delivery jobs, consulting detailed shipping addresses, and updating delivery statuses (In Transit, Delivered, Failed).
  - ➢ **Persistence & Reporting:** Local data persistence and sales/catalog export capabilities using

structured **JSON** and **CSV** formats.

- **Phase 2 (Future Scope - Full Web Application & Database):**
  - ➢ Migration to a decoupled Web Architecture (Frontend HTML/JS + REST API Backend).
  - ➢ Integration of a Relational Database Management System (PostgreSQL / MySQL).
  - ➢ Strict encrypted authentication (bcrypt/Argon2) and external mapping service integration.

What the system will NOT do:

1. Manage employee payroll or salaries.
2. Track the physical condition or maintenance of delivery vehicles.
3. Provide complete enterprise accounting beyond basic sales/inventory reporting in CSV/JSON.
4. Automate supplier stock procurement.

1.3. Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
IEEE	Institute of Electrical and Electronics Engineers
CLI	Command Line Interface (Console execution environment)
JSON / CSV	JavaScript Object Notation / Comma-Separated Values (Formats used for persistence and report generation)
Refactored Functions	Clean Code architectural practice of splitting long procedure blocks into modular, single-responsibility methods.

2. General Description

2.1. Product Perspective & Evolution

The system is designed as an independent management application for a bookstore in Quito, Ecuador. In its current release (Phase 1), the software operates as Java Console that validates all core business rules, object interactions, role permissions, and JSON/CSV data persistence.

This solid object-oriented foundation prepares the project for seamless migration in Phase 2 toward a full web-based platform with REST API endpoints and database storage.

2.2. User Characteristics

User Role	Description	Phase 1 Scope (CLI)	Phase 2 Scope (Web)
Customer	End-user purchasing books.	Direct role access via CLI; catalog search, order creation, payment method selection, and order status tracking.	Web registration/login, interactive shopping cart, online payment gateway, interactive map tracking.
Admin	Bookstore staff managing inventory and operations.	Adding authors, publishers, categories, books, updating inventory levels, low-stock warnings, sales reporting.	Web dashboard with graphical analytics, user account administration, advanced stock management.
Delivery Driver	Logistics personnel fulfilling orders.	Viewing assigned orders, checking customer delivery addresses, updating transit/delivery statuses.	Mobile-friendly web interface with live geolocation routing.

3. Specific Requirements

3.1. Functional Requirements

3.1.1. Core Role-Based Navigation (Clean Code / Refactored Functions)

- **SR-FUN-NAV-01:** The system shall provide a centralized main menu allowing users to select their active role (1. Admin, 2. Customer, 3. Delivery Driver, 0. Exit).
- **SR-FUN-NAV-02:** Each role menu shall be implemented in modular, independent functions (adminMenu(), customerMenu(), deliveryDriverMenu()) obeying Clean Code principles.

3.1.2. Customer Functions

- **SR-FUN-CUST-01:** The system shall allow customers to browse books and search items by title or author.
- **SR-FUN-CUST-02:** The system shall allow customers to select books, specify quantities, provide structured address details (Main street, Secondary street, Sector, Reference), and select a payment method.
- **SR-FUN-CUST-03:** The system shall automatically reduce book stock upon successful order creation.
- **SR-FUN-CUST-04:** Customers shall be able to track order status and view assigned delivery driver information using their Order ID.

3.1.3. Admin Functions

- **SR-FUN-ADMIN-01:** Admins shall be able to register new Authors, Publishers, Categories, and Books.
- **SR-FUN-ADMIN-02:** Admins shall be able to update inventory stock and review low-stock alerts (stock ≤ REORDER_LEVEL).
- **SR-FUN-ADMIN-03:** Admins shall be able to trigger sales and inventory report generation exported in **JSON** and **CSV** formats.

3.1.4. Delivery Driver Functions

- **SR-FUN-DELI-01:** Delivery drivers shall view all delivery assignments created by customer orders.
- **SR-FUN-DELI-02:** Drivers shall inspect detailed shipping address data (using getShippingAddress()) associated with each assignment.
- **SR-FUN-DELI-03:** Drivers shall update order delivery status (e.g., "In Transit", "Delivered", "Failed").

3.2. Data Persistence & Reporting Requirements

- **SR-DAT-01 (JSON Serialization):** The system shall support reading and writing object collections (Books, Orders, Customers) to/from .json files.
- **SR-DAT-02 (CSV Reporting):** The system shall generate comma-separated .csv report files for sales summaries and inventory status.